Report 1: Rudy

Rodrigo Perestrelo, Lennard Last

September 6, 2026

1 Introduction

For this assignment a simple web server was implemented in Erlang.

After ensuring basic functionality, it was discussed how performance can be increased under high load, how fragmented requests can be handled, and how robustness can be improved.

2 Main problems and solutions

2.1 Performance under high load

Since the server would only handle one request at a time, throughput would decrease significantly when simulating computation time by letting the process sleep for 40 ms during reply creation. To improve throughput when facing large request amounts, a pool of request handler processes is created on server startup. Every handler process calls accept on the same shared listening socket. When a connection request arrives at the socket, the first available handler processes it and returns to the worker pool after completion.

2.2 Handling fragmented request data

The first implementation of the server would blindly believe the first string it reads from the socket entails the full request string. Since that can't be guaranteed, the request workflow was changed to keep reading arriving strings from the socket and accumulating the request string until a header terminator (CRLF x2) can be found in it. Only then will the request be parsed. If the parsed body is shorter than the value of the "Content-Length" header (the lack of this header is interpreted as no body to be expected), arriving strings will be read from the socket until the full body has been accumulated. Once the full body has been received or if no body is expected, the handler processes the request and sends a reply.

2.3 Robustness

To improve robustness it was made sure that each handler gets to finish processing its current request before shutting down the server. All handlers are spawned as processes linked to the supervising process using `spawn_link()`. By calling `process_flag(trap_exit, true)` within the supervising process, it will not be killed if a linked process (i.e. one of the handlers) exits abnormally and receives an exit message from the linked process instead. If the server is supposed to shut down, it will close the listening socket, which will cause every handler waiting for an incoming connection request to return and send an exit signal to the supervisor. Once all handlers have exited gracefully, the supervising process will also return. This supervisor pattern also made it easy to implement logic for restarting crashed handlers, since they will send an exit message with a reason other than `normal`.

3 Evaluation

A simple benchmark was implemented in Erlang that measures the time it takes for 100 requests to be processed by the server, to test performance under high load. Since barely any computation is being done when processing a request, it was simulated using `timer:sleep(40)` in the function creating the reply. The results of these benchmarks in various configurations can be seen in Figure 1. The average of five benchmarks run in succession was used as the baseline for this test. It is shown by the leftmost box in Figure 1. When running five benchmarks in parallel without pooling, the average time increased drastically, since all requests would have to wait for the server to parse the request, sleep for 40 ms and then send the reply. Adding handlers to the pool decreased the average time and a pool of 5 handlers showed average times similar to the baseline since it means a 1:1 allocation of requests to handlers. Although BEAM supports symmetric multiprocessing, the average time did not change by using more scheduler threads. The reason being that calling `timer:sleep(40)` makes the scheduler skip over the process until it is done sleeping, so it is not a very accurate way of simulating CPU load. To highlight this, the same benchmarks were run using a function that actually occupies the scheduler for approximately 30 ms. The rightmost group of boxes visualizing the results of these benchmarks shows very clearly that throughput only improves when using multiple schedulers to host the handler processes in the pool. The main insight of these tests is that the performance scales linearly as long as the number of handlers in the pool does not exceed the number of schedulers running on the CPU, otherwise the performance depends on the amount of CPU-bound work being done by each handler process.

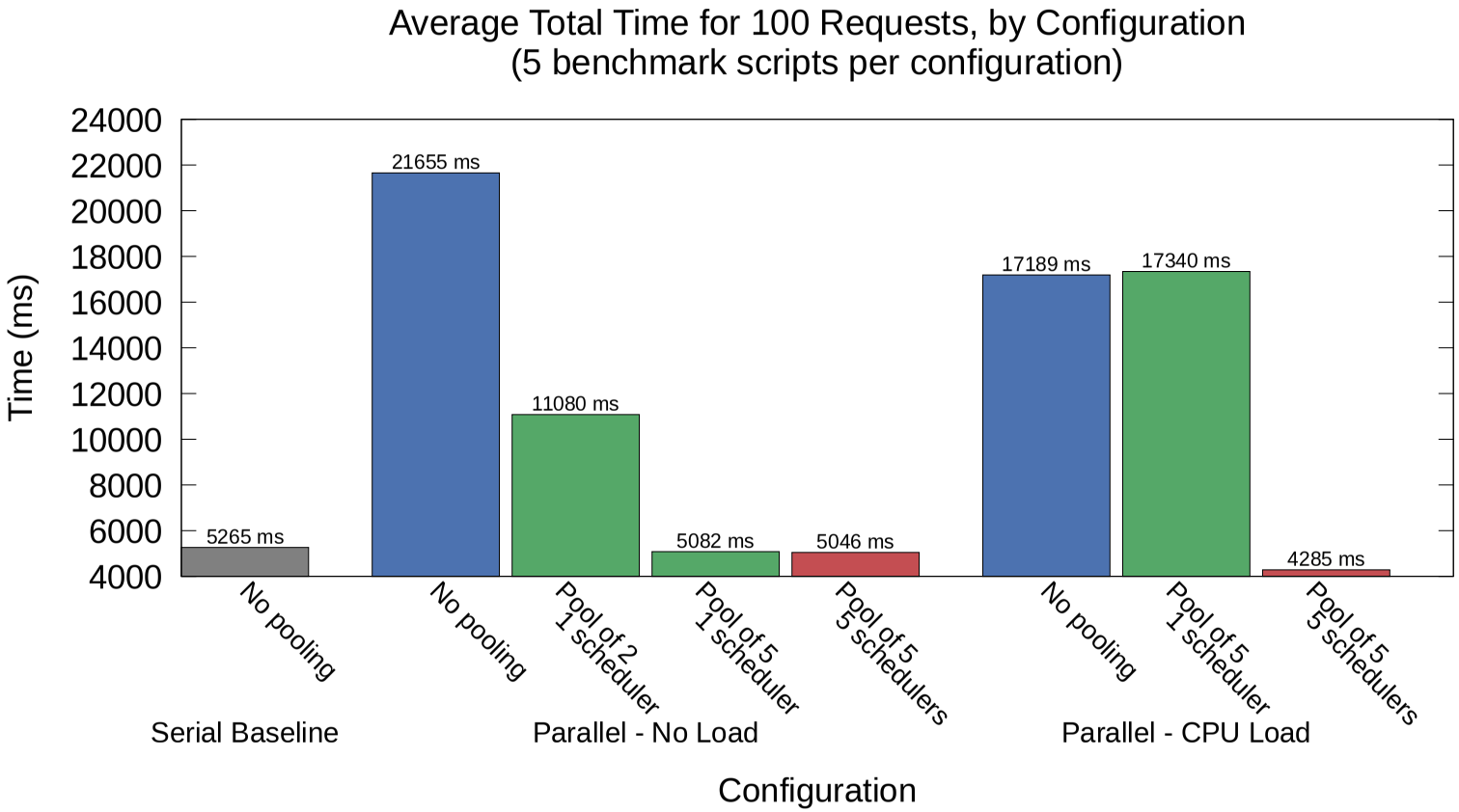


Figure 1: Benchmark results

4 Conclusions

This assignment was a good example of how the scalability and robustness of a system can be improved by using clever design patterns like a prefork worker pool or a supervisor. It also showed quite clearly that in spite of all cleverness, scalability is still tied to the hardware performing the computation.